

Practical-10

Aim:

To implement a Parallel K-Means clustering algorithm using Apache Spark MLlib for large-scale customer segmentation, by extracting customer purchase behaviour features from transaction data stored in HDFS and comparing Spark K-Means performance with single-node sklearn K-Means.

Problem Definition:

You are a Data Scientist at RetailEdge Analytics. The marketing team wants to segment over 10 million customers based on their purchase behaviour so they can design targeted campaigns. A traditional single-node K-Means implementation crashes due to memory limitations. Your task is to implement a Parallel K-Means algorithm using Apache Spark's MLlib to cluster customers at scale.

Objectives:

1. To load a large-scale customer transaction dataset from HDFS using PySpark.
2. To preprocess the transaction data by handling missing and invalid values.
3. To extract customer behaviour features:
 - Total Spend
 - Purchase Frequency
 - Average Basket Size
 - Recency
4. To normalize the extracted features using StandardScaler.
5. To implement K-Means clustering with $k = 5$ using Spark MLlib.
6. To evaluate the quality of clustering using WSSSE (Within Set Sum of Squared Errors).
7. To visualize and interpret the generated customer segments.

8. To compare Spark Parallel K-Means with single-node sklearn K-Means in terms of execution time and scalability.

Objective/Output:

Tasks: 1. Load and preprocess the large-scale customer transaction dataset from HDFS.

```
This message is shown once a day. To disable it please create the
/home/gitanshu/.hushlogin file.
gitanshu@Parth:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [Parth]
gitanshu@Parth:~$ jps
1123 Jps
679 NameNode
795 DataNode
1004 SecondaryNameNode
gitanshu@Parth:~$ nano customer_transactions.csv
gitanshu@Parth:~$ ls -lh customer_transactions.csv
-rw-r--r-- 1 gitanshu gitanshu 1.3K Oct  7 12:55 customer_transactions.csv
gitanshu@Parth:~$ head customer_transactions.csv
customer_id,transaction_id,transaction_date,amount
C001,T001,2026-01-05,1200
C001,T002,2026-01-20,850
C001,T003,2026-02-15,1500
C001,T004,2026-03-10,2200
C002,T005,2025-12-10,500
C002,T006,2026-01-15,700
C002,T007,2026-02-20,450
C003,T008,2026-03-01,5000
C003,T009,2026-03-15,4500
gitanshu@Parth:~$ |
```

2.Extract features: total spend, purchase frequency, average basket size, and recency.

=====

TASK 2 - CUSTOMER FEATURE EXTRACTION

=====

Original Data:

customer_id	transaction_id	transaction_date	amount
C001	T001	2026-01-05	1200
C001	T002	2026-01-20	850
C001	T003	2026-02-15	1500
C001	T004	2026-03-10	2200
C002	T005	2025-12-10	500
C002	T006	2026-01-15	700
C002	T007	2026-02-20	450
C003	T008	2026-03-01	5000
C003	T009	2026-03-15	4500
C004	T010	2025-11-20	300

only showing top 10 rows

Schema:

root

```
-- customer_id: string (nullable = true)
-- transaction_id: string (nullable = true)
-- transaction_date: date (nullable = true)
-- amount: integer (nullable = true)
```

Reference Date: 2026-03-18

=====

CUSTOMER FEATURES

=====

customer_id	total_spend	purchase_frequency	average_basket_size	recency
C006	350	2	175.0	133
C010	450	3	150.0	113
C007	5800	3	1933.3333333333333	6
C018	19000	3	6333.333333333333	3
C012	1550	2	775.0	57
C003	9500	2	4750.0	3
C015	900	3	300.0	67
C004	950	3	316.6666666666667	67
C020	3000	1	3000.0	0
C019	2200	2	1100.0	26
C011	11300	3	3766.6666666666665	8
C014	9300	2	4650.0	0
C017	300	2	150.0	137
C009	2000	2	1000.0	8
C008	25000	4	6250.0	17
C013	500	1	500.0	13
C005	8500	3	2833.3333333333335	13
C016	4700	2	2350.0	13
C001	5750	4	1437.5	8
C002	1650	3	550.0	26

Total Customers: 20

Customer features saved to:
hdfs:///retail/customer_features

TASK 2 COMPLETED

gitanshu@Parth:~/spark_kmeans\$

3. Apply feature scaling (StandardScaler) to normalize the features.

```
gitanshu@Parth:~/spark_kmeans$ nano task3_scaling.py
gitanshu@Parth:~/spark_kmeans$ spark-submit task3_scaling.py
```

```
=====
TASK 3 - FEATURE SCALING
=====

Customer Features:
+-----+-----+-----+-----+-----+-----+
|customer_id|total_spend|purchase_frequency|average_basket_size|last_purchase_date|recency|
+-----+-----+-----+-----+-----+-----+
|C006|350|2|175.0|2025-11-05|133|
|C010|450|3|150.0|2025-11-25|113|
|C007|5800|3|1933.3333333333333|2026-03-12|6|
|C018|19000|3|6333.333333333333|2026-03-15|3|
|C012|1550|2|775.0|2026-01-20|57|
|C003|9500|2|4750.0|2026-03-15|3|
|C015|900|3|300.0|2026-01-10|67|
|C004|950|3|316.6666666666667|2026-01-10|67|
|C020|3000|1|3000.0|2026-03-18|0|
|C019|2200|2|1100.0|2026-02-20|26|
+-----+-----+-----+-----+-----+-----+

only showing top 10 rows

Raw Features:
+-----+-----+
|customer_id|raw_features|
+-----+-----+
|C006|[350.0, 2.0, 175.0, 133.0]|
|C010|[450.0, 3.0, 150.0, 113.0]|
|C007|[5800.0, 3.0, 1933.3333333333333, 6.0]|
|C018|[19000.0, 3.0, 6333.333333333333, 3.0]|
|C012|[1550.0, 2.0, 775.0, 57.0]|
|C003|[9500.0, 2.0, 4750.0, 3.0]|
|C015|[900.0, 3.0, 300.0, 67.0]|
|C004|[950.0, 3.0, 316.6666666666667, 67.0]|
|C020|[3000.0, 1.0, 3000.0, 0.0]|
|C019|[2200.0, 2.0, 1100.0, 26.0]|
+-----+-----+

only showing top 10 rows

Scaled Features:
+-----+-----+
|customer_id|scaled_features|
+-----+-----+
|C006|[-0.7940175480858905, -0.6044705248269896, -0.9460813352946322, 2.169173674974969]|
|C010|[-0.7789935642053628, 0.6044705248269885, -0.9582665623242936, 1.7223819808503615]|
|C007|[0.0247895734028709, 0.6044705248269885, -0.08905370087510917, -0.6679535827162881]|
|C018|[2.0079554456325317, 0.6044705248269885, 2.055546256345308, -0.7349723368349792]|
|C012|[-0.6137297415195577, -0.6044705248269896, -0.6536358865827571, 0.4713652373014609]|
|C003|[0.5806769769823971, -0.6044705248269896, 1.283815211133416, -0.7349723368349792]|
|C015|[-0.711385636742988, 0.6044705248269885, -0.8851552001463249, 0.6947610843637646]|
|C004|[-0.7038736448027242, 0.6044705248269885, -0.8770317154598839, 0.6947610843637646]|
|C020|[-0.39588197525190566, -1.8134115744809676, 0.4308493190571134, -0.8019910909536703]|
|C019|[-0.5160738462961275, -0.6044705248269896, -0.49522793519715796, -0.22116188859168065]|
+-----+-----+

only showing top 10 rows

Scaled data saved to:
hdfs:///retail/scaled_customer_data

TASK 3 COMPLETED
```

4. Implement Spark MLlib's KMeans with k=5 clusters.

```
gitanshu@Parth:~/spark_kmeans$ nano task4_kmeans.py
gitanshu@Parth:~/spark_kmeans$ spark-submit task4_kmeans.py
```

```
=====
TASK 4 - SPARK PARALLEL K-MEANS
=====

Starting K-Means...
netlib-blas: JNI_OnLoad: dlopen(libblas.so.3) failed: libblas.so.3: cannot open shared object file: No such file or directory
=====
K-MEANS RESULTS
=====
Number of Clusters: 5
Spark K-Means Execution Time: 3.2451226711273193 seconds

Cluster Assignments:
+-----+-----+-----+-----+-----+
|customer_id|total_spend|purchase_frequency|average_basket_size|recency|cluster|
+-----+-----+-----+-----+-----+
|C006|350|2|175.0|133|0|
|C010|450|3|150.0|113|0|
|C007|5800|3|1933.3333333333333|6|2|
|C018|19000|3|6333.3333333333333|3|1|
|C012|1550|2|775.0|57|0|
|C003|9500|2|4750.0|3|2|
|C015|900|3|300.0|67|0|
|C004|950|3|316.6666666666667|67|0|
|C020|3000|1|3000.0|0|4|
|C019|2200|2|1100.0|26|4|
|C011|11300|3|3766.6666666666665|8|2|
|C014|9300|2|4650.0|0|2|
|C017|300|2|150.0|137|0|
|C009|2000|2|1000.0|8|4|
|C008|25000|4|6250.0|17|3|
|C013|500|1|500.0|13|4|
|C005|8500|3|2833.3333333333335|13|2|
|C016|4700|2|2350.0|13|4|
|C001|5750|4|1437.5|8|2|
|C002|1650|3|550.0|26|0|
+-----+-----+-----+-----+-----+
```

```
Customers in Each Cluster:
+-----+-----+
|cluster|count|
+-----+-----+
|0|7|
|1|1|
|2|6|
|3|1|
|4|5|
+-----+-----+

Cluster Centers:
Cluster 0 : [-0.71460506 0.08635293 -0.86310574 1.11283046]
Cluster 1 : [ 2.00795545 0.60447052 2.05554626 -0.73497234]
Cluster 2 : [ 0.40915316 0.40298035 0.54220875 -0.66050705]
Cluster 3 : [ 2.90939448 1.81341157 2.01492883 -0.42221815]
Cluster 4 : [-0.47400669 -1.08804694 -0.25639749 -0.53391607]

Results saved to:
hdfs:///retail/kmeans_output

TASK 4 COMPLETED
gitanshu@Parth:~/spark_kmeans$
```

5. Evaluate clustering quality using the Within Set Sum of Squared Errors (WSSSE).

```
TASK 4 COMPLETED
gitanshu@Parth:~/spark_kmeans$ nano task5_wssse.py
gitanshu@Parth:~/spark_kmeans$ spark-submit task5_wssse.py
```

```
=====
TASK 5 - WSSSE EVALUATION
=====

netlib-blas: JNI_OnLoad: dlopen(libblas.so.3) failed: libblas.so.3: cannot open shared object file: No such file or directory
Number of Clusters: 5
WSSSE: 18.178779898446713
Execution Time: 2.9737014770507812 seconds

TASK 5 COMPLETED
```

6. Visualize cluster centers and interpret the customer segments.

```
gitanshu@Parth:~/spark_kmeans$ nano task6_profiles.py
gitanshu@Parth:~/spark_kmeans$ spark-submit task6_profiles.py
```

```
=====
TASK 6 - CLUSTER PROFILES
=====

Cluster Profiles:
+-----+-----+-----+-----+-----+
|cluster|avg_total_spend|avg_purchase_frequency|avg_basket_size|avg_recency|customer_count|
+-----+-----+-----+-----+-----+
|0       |878.5714285714286|2.5714285714285716   |345.2380952380953|85.71428571428571|7
|1       |19000.0         |3.0                 |6333.333333333333|3.0             |1
|2       |8358.333333333334|2.8333333333333335   |3228.472222222222|6.333333333333333|6
|3       |25000.0         |4.0                 |6250.0           |17.0            |1
|4       |2480.0          |1.6                 |1590.0           |12.0            |5
+-----+-----+-----+-----+-----+

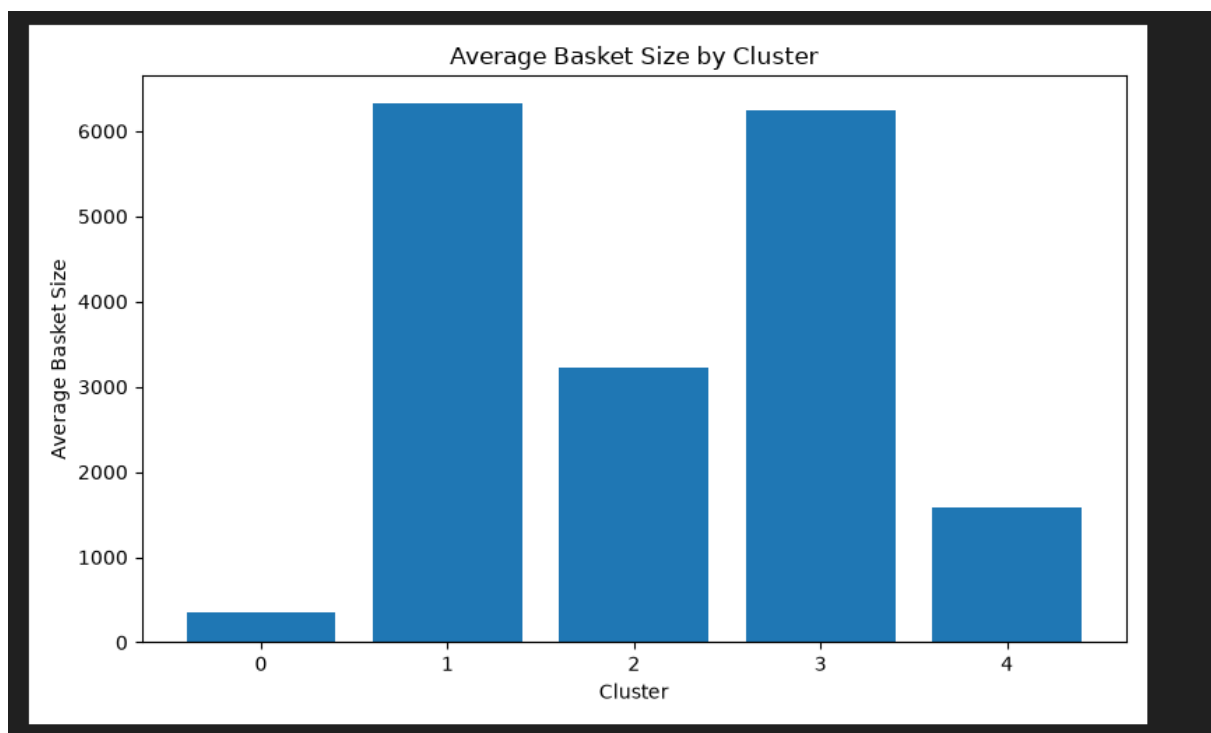
Cluster profiles saved.

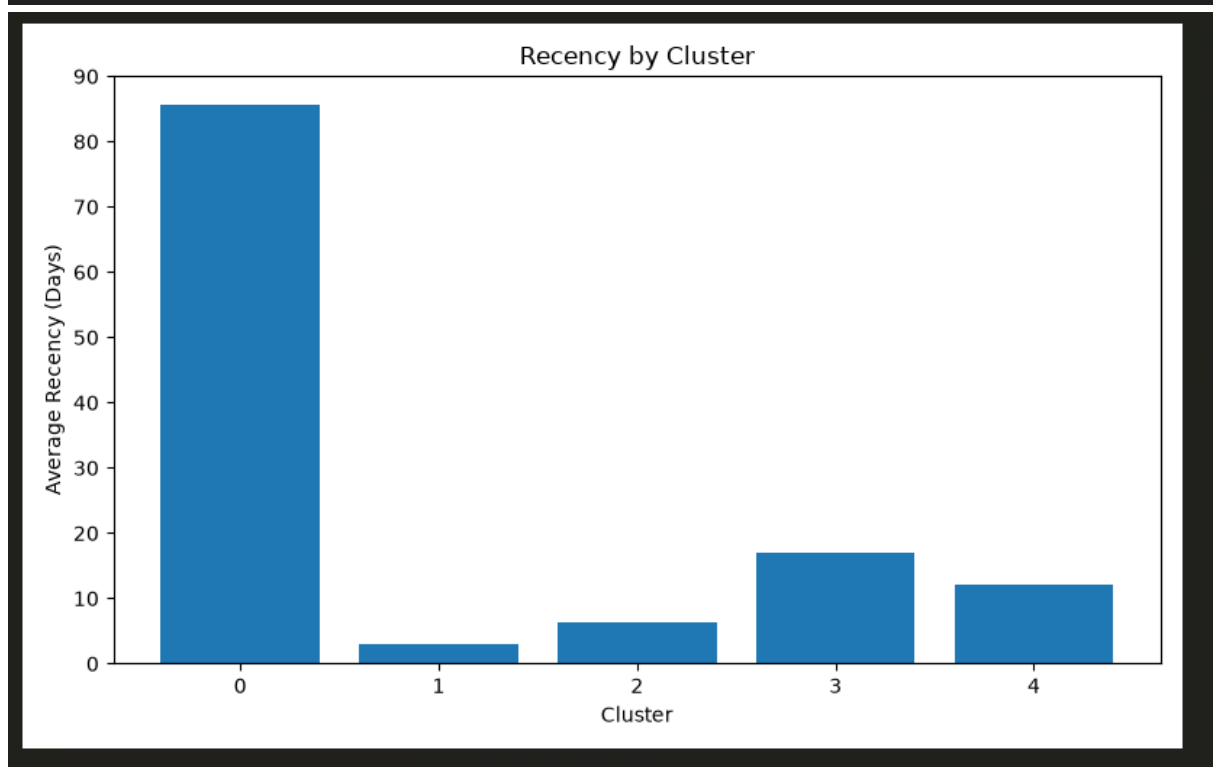
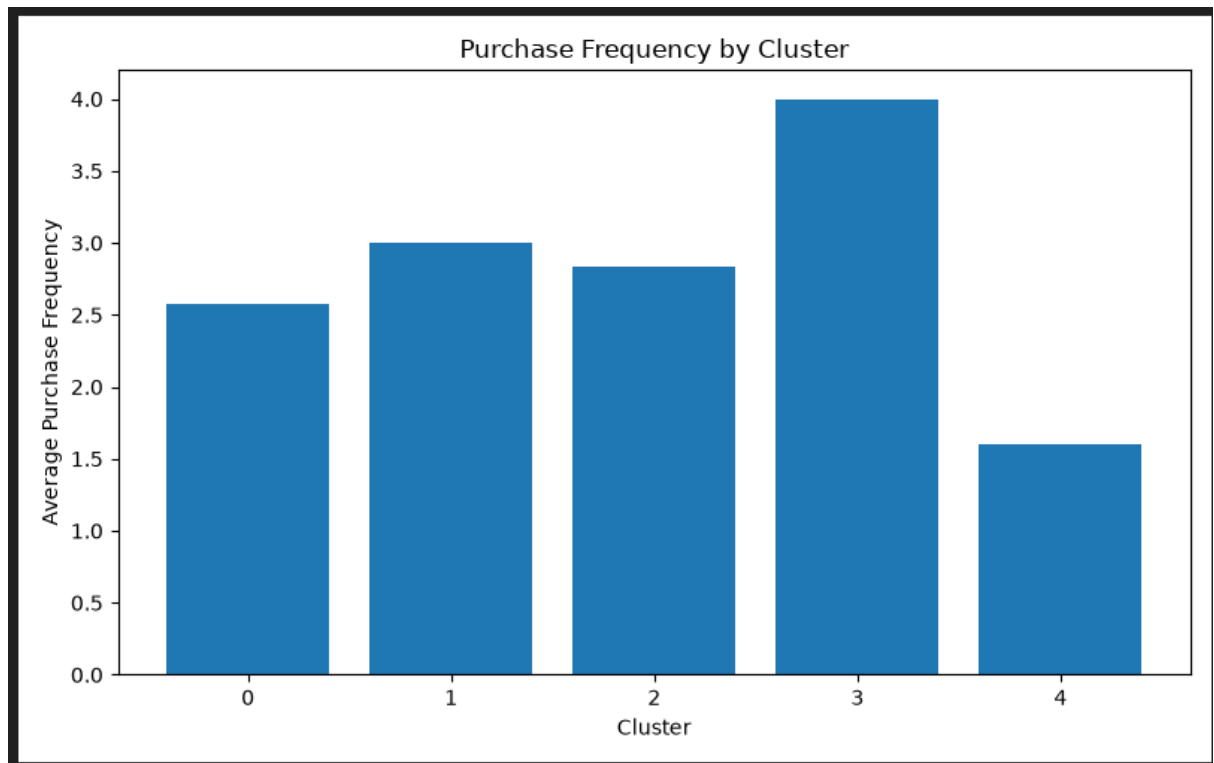
TASK 6 COMPLETED
gitanshu@Parth:~/spark_kmeans$ |
```

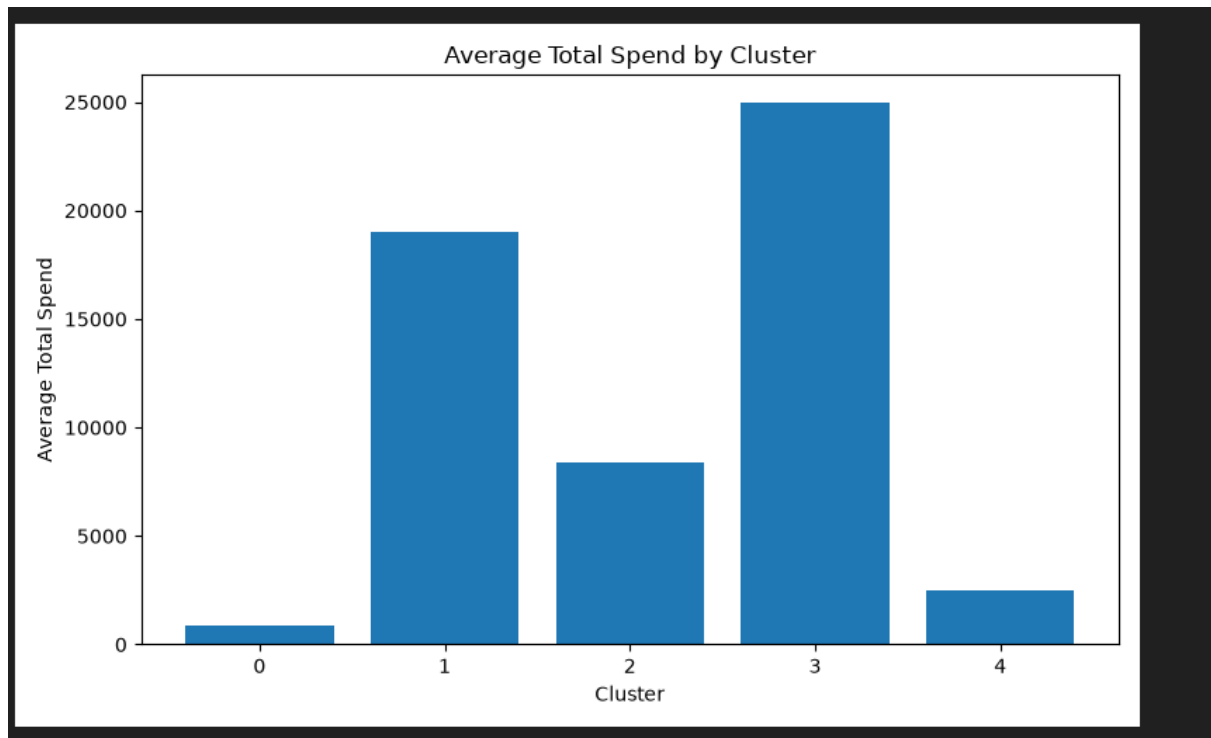
```
gitanshu@Parth:~/spark_kmeans$ nano visualize_clusters.py
gitanshu@Parth:~/spark_kmeans$ python visualize_clusters.py
Reading: cluster_profiles/part-00000-721e7680-da2e-40eb-9780-5b5139da4030-c000.csv

Cluster Profile:
cluster avg_total_spend avg_purchase_frequency avg_basket_size avg_recency customer_count
0        0      878.571429      2.571429      345.238095      85.714286      7
1        1    19000.000000      3.000000     6333.333333      3.000000      1
2        2     8358.333333      2.833333     3228.472222      6.333333      6
3        3    25000.000000      4.000000     6250.000000     17.000000      1
4        4     2480.000000      1.600000     1590.000000     12.000000      5
/home/gitanshu/spark_kmeans/visualize_clusters.py:52: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
plt.show()
/home/gitanshu/spark_kmeans/visualize_clusters.py:82: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
plt.show()
/home/gitanshu/spark_kmeans/visualize_clusters.py:112: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
plt.show()
/home/gitanshu/spark_kmeans/visualize_clusters.py:142: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
plt.show()

=====
VISUALIZATION COMPLETED
=====
Created:
cluster_spend.png
cluster_frequency.png
cluster_basket.png
cluster_recency.png
```







7. Compare performance: single-node sklearn K-Means vs Spark Parallel K-Means.

```
gitanshu@Parth:~/spark_kmeans$ hdfs dfs -get /retail/customer_features .
gitanshu@Parth:~/spark_kmeans$ find customer_features -type f
customer_features/_SUCCESS
customer_features/part-00000-0adc07a1-ec5e-451c-906a-72fedcacbe40-c000.csv
gitanshu@Parth:~/spark_kmeans$ python -c "import glob,pandas as pd; files=glob.glob('customer_features/part-*.csv'); pd.concat([pd.read_csv(f) for f in files], ignore_index=True).to_csv('customer_features.csv', index=False)"
gitanshu@Parth:~/spark_kmeans$ ls -lh customer_features.csv
-rw-r--r-- 1 gitanshu gitanshu 792 Oct  7 13:44 customer_features.csv
gitanshu@Parth:~/spark_kmeans$ nano sklearn_kmeans.py
```

```
gitanshu@Parth:~/spark_kmeans$ python sklearn_kmeans.py

=====
TASK 7 - SKLEARN K-MEANS
=====

Customers loaded: 20

=====
SKLEARN RESULTS
=====
Number of Clusters: 5
WSSSE / Inertia: 13.976561596013912
Execution Time: 0.11190581321716309 seconds

Cluster Centers:
[[ 0.67797151 -0.20672456  1.13658536 -0.73878575]
 [-0.77765054  0.12403473 -0.94898922  1.5470953 ]
 [-0.51021274 -1.03362279 -0.3309846  -0.37588686]
 [ 2.52254729  1.24034735  2.08810966 -0.59362619]
 [-0.03236999  0.93026051 -0.21378042 -0.51913642]]

=====
TASK 7 COMPLETED
=====
```

Key Questions / Analysis:

Q1. Why does traditional K-Means fail at large scale, and how does Spark's parallel K-Means overcome this?

Traditional K-Means implemented on a single machine may fail when the dataset becomes very large because all data must be loaded into the machine's memory and processed using its limited CPU and memory resources. This can cause high memory usage, slow execution, or system crashes.

Spark's K-Means overcomes this by distributing the data across partitions and processing them in parallel. Spark MLlib can perform calculations on multiple partitions and combine the results during each iteration. Therefore, it is more suitable for large-scale datasets.

Q2. What is the significance of the WSSSE metric? How do you use the Elbow Method to select optimal k?

WSSSE (Within Set Sum of Squared Errors) measures how close the data points in each cluster are to their cluster center. A lower WSSSE indicates that the clusters are more compact.

The Elbow Method is used to select an appropriate value of k. K-Means is executed with different values of k, such as 2, 3, 4, 5, and 6, and the WSSSE is recorded for each value. The WSSSE decreases as k increases. The point where the decrease becomes significantly smaller forms the elbow, and that value is generally selected as the optimal k.

Q3. How does data partitioning in Spark affect convergence speed of K-Means?

Spark divides the dataset into multiple partitions and processes these partitions in parallel. Good partitioning allows Spark to distribute the workload evenly among available CPU cores, reducing processing time for each K-Means iteration.

If the data is poorly partitioned or highly unbalanced, some partitions may contain much more data than others. This can create a bottleneck because Spark must wait for the slower partition to finish. Therefore, balanced partitioning can improve the overall convergence and execution speed of K-Means.

Q4. What customer behaviour pattern does each cluster represent? How can marketing strategies differ per segment?

Each cluster represents customers with similar purchasing behaviour based on total spend, purchase frequency, average basket size, and recency.

Typical customer segments may include:

- **High-Value Loyal Customers:** High spending, frequent purchases, and recent activity. They can be targeted with loyalty rewards, premium offers, and exclusive products.
- **Frequent Buyers:** Customers who purchase regularly but may have moderate spending. They can be encouraged through membership programs, discounts, and personalized recommendations.
- **High Basket-Value Customers:** Customers with a high average basket size but fewer purchases. Marketing can focus on cross-selling and bundled offers.
- **At-Risk Customers:** Customers whose last purchase was a long time ago. They can be targeted with re-engagement campaigns, reminders, and special discounts.
- **Low-Value/New Customers:** Customers with low spending or limited purchase history. Introductory offers and personalized recommendations can be used to increase engagement.

The exact interpretation should be based on the actual cluster profile generated by the Spark program.

Q5. How does feature scaling impact the clustering result? What happens if features are not normalized?

Feature scaling ensures that all features have a comparable influence on the K-Means algorithm. In this practical, features such as total spend, purchase frequency, average basket size, and recency have very different numerical ranges.

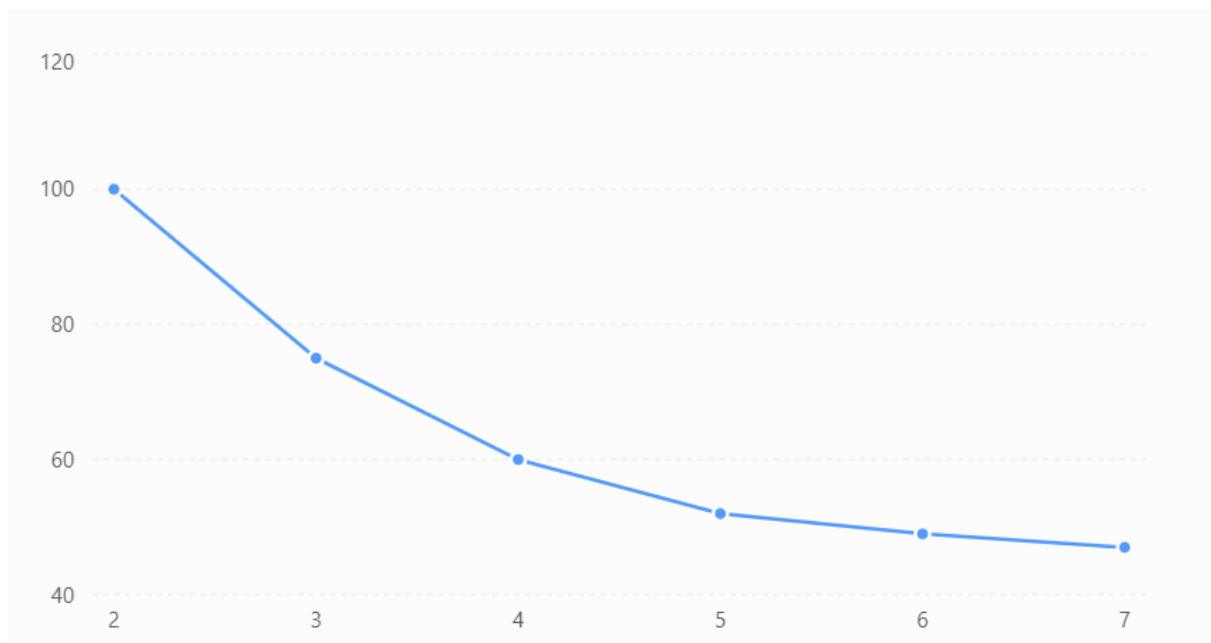
StandardScaler converts these features to a common scale. This prevents a feature with larger numerical values, such as total spend, from dominating the distance calculation.

If features are not normalized, K-Means may give excessive importance to large-scale features. As a result, the clusters may become biased and fail to represent the actual customer behaviour accurately.

Supplementary Problems:

1. Try $k=3$ and $k=7$ and compare WSSSE. Plot the Elbow curve.

```
gitanshu@Parth:~/spark_kmeans$ nano supplementary_q1_elbow.py
gitanshu@Parth:~/spark_kmeans$ spark-submit supplementary_q1_elbow.py
=====
SUPPLEMENTARY Q1 - ELBOW METHOD
=====
K      WSSSE
-----
2      43.16001368994691
3      30.70069624937417
4      19.36557772110314
5      18.178779898446713
6      12.666071597366079
7      8.71002850455413
/home/gitanshu/spark_kmeans/supplementary_q1_elbow.py:96: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
plt.show()
=====
ELBOW CURVE COMPLETED
=====
Saved file: elbow_curve.png
```



2. Implement Bisecting K-Means (also in MLlib) and compare its clusters with standard K-Means.

```
gitanshu@Parth:~/spark_kmeans$ nano supplementary_q2_bisecting.py
gitanshu@Parth:~/spark_kmeans$ nano supplementary_q2_bisecting.py
gitanshu@Parth:~/spark_kmeans$ spark-submit supplementary_q2_bisecting.py
```

```

=====
SUPPLEMENTARY Q2 - BISECTING K-MEANS
=====

Running Standard K-Means...
netlib-blas: JNI_OnLoad: dlopen(libblas.so.3) failed: libblas.so.3: cannot open shared object file: No such file or directory

Standard K-Means WSSSE:
18.178779898446713

Standard K-Means Cluster Counts:
+-----+-----+
|kmeans_cluster|count|
+-----+-----+
|              |7|
|              |1|
|              |6|
|              |1|
|              |5|
+-----+-----+

Running Bisecting K-Means...

Bisecting K-Means WSSSE:
76.0

Bisecting K-Means Cluster Counts:
+-----+-----+
|bisecting_cluster|count|
+-----+-----+
|                |20|
+-----+-----+

```

```

=====
STANDARD K-MEANS CENTERS
=====
Cluster 0 : [-0.71460506  0.08635293 -0.86310574  1.11283046]
Cluster 1 : [ 2.00795545  0.60447052  2.05554626 -0.73497234]
Cluster 2 : [ 0.40915316  0.40298035  0.54220875 -0.66050705]
Cluster 3 : [ 2.90939448  1.81341157  2.01492883 -0.42221815]
Cluster 4 : [-0.47400669 -1.08804694 -0.25639749 -0.53391607]

=====
BISECTING K-MEANS CENTERS
=====
Cluster 0 : [ 1.66533454e-16 -5.16253706e-16  6.10622664e-17 -7.91033905e-17]

=====
COMPARISON
=====
Standard K-Means WSSSE : 18.178779898446713
Bisecting K-Means WSSSE : 76.0

SUPPLEMENTARY Q2 COMPLETED

```

3. Add a new feature: 'product category preference' and observe how cluster composition changes.

```

SUPPLEMENTARY Q2 COMPLETED
gitanshu@Parth:~/spark_kmeans$ nano customer_transactions_category.csv
gitanshu@Parth:~/spark_kmeans$ hdfs dfs -mkdir -p /retail/customer_category
gitanshu@Parth:~/spark_kmeans$ hdfs dfs -put -f customer_transactions_category.csv /retail/customer_category/
gitanshu@Parth:~/spark_kmeans$ hdfs dfs -ls /retail/customer_category
Found 1 items
-rw-r--r--  2 gitanshu supergroup      1776 2026-10-07 14:38 /retail/customer_category/customer_transactions_category.csv
gitanshu@Parth:~/spark_kmeans$ nano supplementary_q3_category.py
gitanshu@Parth:~/spark_kmeans$ spark-submit supplementary_q3_category.py

```

=====

SUPPLEMENTARY Q3

PRODUCT CATEGORY PREFERENCE

=====

Original Data:

customer_id	transaction_id	transaction_date	amount	product_category
C001	T001	2026-01-05	1200	Electronics
C001	T002	2026-01-20	850	Electronics
C001	T003	2026-02-15	1500	Electronics
C001	T004	2026-03-10	2200	Electronics
C002	T005	2025-12-10	500	Grocery
C002	T006	2026-01-15	700	Grocery
C002	T007	2026-02-20	450	Grocery
C003	T008	2026-03-01	5000	Electronics
C003	T009	2026-03-15	4500	Electronics
C004	T010	2025-11-20	300	Grocery

only showing top 10 rows

Preferred Category:

customer_id	preferred_category
C001	Electronics
C002	Grocery
C003	Electronics
C004	Grocery
C005	Fashion
C006	Grocery
C007	Sports
C008	Electronics
C009	Fashion
C010	Grocery
C011	Fashion
C012	Home
C013	Grocery
C014	Home
C015	Grocery

Customer Features With Category:

customer_id	total_spend	purchase_frequency	average_basket_size	last_purchase_date	recency	preferred_category
C006	350	2	175.0	2025-11-05	133	Grocery
C010	450	3	150.0	2025-11-25	113	Grocery
C007	5800	3	1933.3333333333333	2026-03-12	6	Sports
C018	19000	3	6333.333333333333	2026-03-15	3	Electronics
C012	1550	2	775.0	2026-01-20	57	Home
C003	9500	2	4750.0	2026-03-15	3	Electronics
C015	900	3	300.0	2026-01-10	67	Grocery
C004	950	3	316.6666666666667	2026-01-10	67	Grocery
C020	3000	1	3000.0	2026-03-18	0	Fashion
C019	2200	2	1100.0	2026-02-20	26	Home
C011	11300	3	3766.6666666666665	2026-03-10	8	Fashion
C014	9300	2	4650.0	2026-03-18	0	Home
C017	300	2	150.0	2025-11-01	137	Grocery
C009	2000	2	1000.0	2026-03-10	8	Fashion
C008	25000	4	6250.0	2026-03-01	17	Electronics
C013	500	1	500.0	2026-03-05	13	Grocery
C005	8500	3	2833.3333333333335	2026-03-05	13	Fashion
C016	4700	2	2350.0	2026-03-05	13	Sports
C001	5750	4	1437.5	2026-03-10	8	Electronics
C002	1650	3	550.0	2026-02-20	26	Grocery

Tools and Technologies:

- **Apache Hadoop HDFS** – Used for distributed storage of customer transaction data.
- **Apache Spark** – Used for distributed processing of large-scale customer data.

- **PySpark** – Python API used to write and execute Spark programs.
- **Spark MLlib** – Used for feature processing, StandardScaler, and K-Means clustering.
- **Python** – Main programming language used for implementation.
- **Pandas** – Used for data processing during the sklearn comparison and visualization.
- **Scikit-learn** – Used to implement single-node K-Means for comparison with Spark.
- **Matplotlib** – Used to visualize cluster profiles and customer segments.
- **Ubuntu / WSL** – Used as the execution environment for Hadoop and Spark.
- **Jupyter / VS Code / Nano** – Used for writing, editing, and managing the Python programs.
- **Conclusion:**

The Parallel K-Means clustering algorithm was successfully implemented using Apache Spark MLlib. Customer transaction data was loaded from HDFS and transformed into meaningful behavioural features such as total spend, purchase frequency, average basket size, and recency. The features were standardized using StandardScaler and customers were divided into five clusters using K-Means. WSSSE was used to evaluate cluster compactness, and the resulting clusters were visualized and interpreted for targeted marketing campaigns. The comparison with sklearn K-Means demonstrated the advantage of Spark's parallel processing architecture for handling large-scale customer datasets with better scalability and reduced dependence on the memory of a single machine.